



Documento Técnico: Sistema de Validaciones para Dominio de Inspección y Fabricación

1. Introducción

Este documento describe el sistema de validaciones implementado en las clases de dominio para el API Rest del SID del LAPEM.

2. Arquitectura de Validación

2.1. Patrón de Diseño

Las clases utilizan el patrón **Domain Object Validation** donde cada entidad es responsable de validar su propio estado interno mediante la implementación del método `IsValid()`.

2.2. Clase Base

```
public abstract class DomainObject
{
    public abstract void IsValid();
}
```

3. Clases de Dominio y sus Validaciones

3.1. EstadoSID

Propósito: Representa estados del sistema de inspección y documentación.

Validaciones Implementadas:

- El campo Estado debe contener valores predefinidos: "EN_ESPERA" o "EN_PRUEBAS"
- Validación mediante lista de valores permitidos

Código de Validación:

```
var estadosValidos = new List<string>() { "EN_ESPERA", "EN_PRUEBAS" };
if (string.IsNullOrEmpty(Estado) || !estadosValidos.Contains(Estado))
    throw new IndexOutOfRangeException($"El estado {Estado} no es válido.");
```

3.2. ExpedienteInspeccion

Propósito: Gestiona expedientes de inspección con muestras y pruebas asociadas.

Validaciones por Estado:

Estado "PENDIENTE":

- Si el tipo de muestreo es "100%", calcula automáticamente el tamaño de muestra

Estado "EN_PROCESO":

- Verifica coincidencia entre tamaño de muestra y número de muestras capturadas
- Valida que los valores de referencia correspondan al producto de la orden de fabricación



Estado "TERMINADO":

- Valida que el resultado sea "SATISFACTORIO" o "NO_SATISFACTORIO"

Validaciones Comunes:

- Tamaño de muestra mayor que cero
- Orden de fabricación requerida

3.3. OrdenFabricacion

Propósito: Representa órdenes de fabricación con sus detalles.

Validaciones Implementadas:

- Clave y lote de fabricación no vacíos
- Producto requerido
- Al menos un detalle de fabricación
- Cantidades a fabricar mayores a cero
- Tipos de contrato válidos: "ContratoCFE", "ContratoCFEConGarantia", "ContratoParticular"

3.4. OtrasPruebasYDocumentos

Propósito: Gestiona documentos y pruebas adicionales.

Validaciones Implementadas:

- Tipo de documento debe ser: "CertificadoMaterial", "Prueba Rutina" u "Otro"

3.5. Producto

Propósito: Define productos con sus características y pruebas asociadas.

Validaciones Implementadas:

- Tipo de fabricación: "LOTE" o "SERIE"
- Norma y prototipo requeridos
- Al menos una prueba definida

3.6. Prueba

Propósito: Define pruebas que se aplican a los productos.

Validaciones Implementadas:

- Tipo de prueba: "RUTINA" o "ACEPTACION"
- Tipo de resultado: "VALOR_REFERENCIA" o "PASA/NO-PASA"
- Estatus: "ACTIVA" o "INACTIVA"

3.7. ResultadoPrueba

Propósito: Registra resultados de pruebas realizadas.

Validaciones Implementadas:

- Todos los campos requeridos no nulos
- Valor medido mayor o igual a cero
- Número de intento mayor a cero
- Coherencia entre prueba y valor de referencia



- Cálculo automático de resultado para tipo "VALOR_REFERENCIA"
- Validación de resultado final

3.8. ValorReferencia

Propósito: Define valores de referencia para las pruebas.

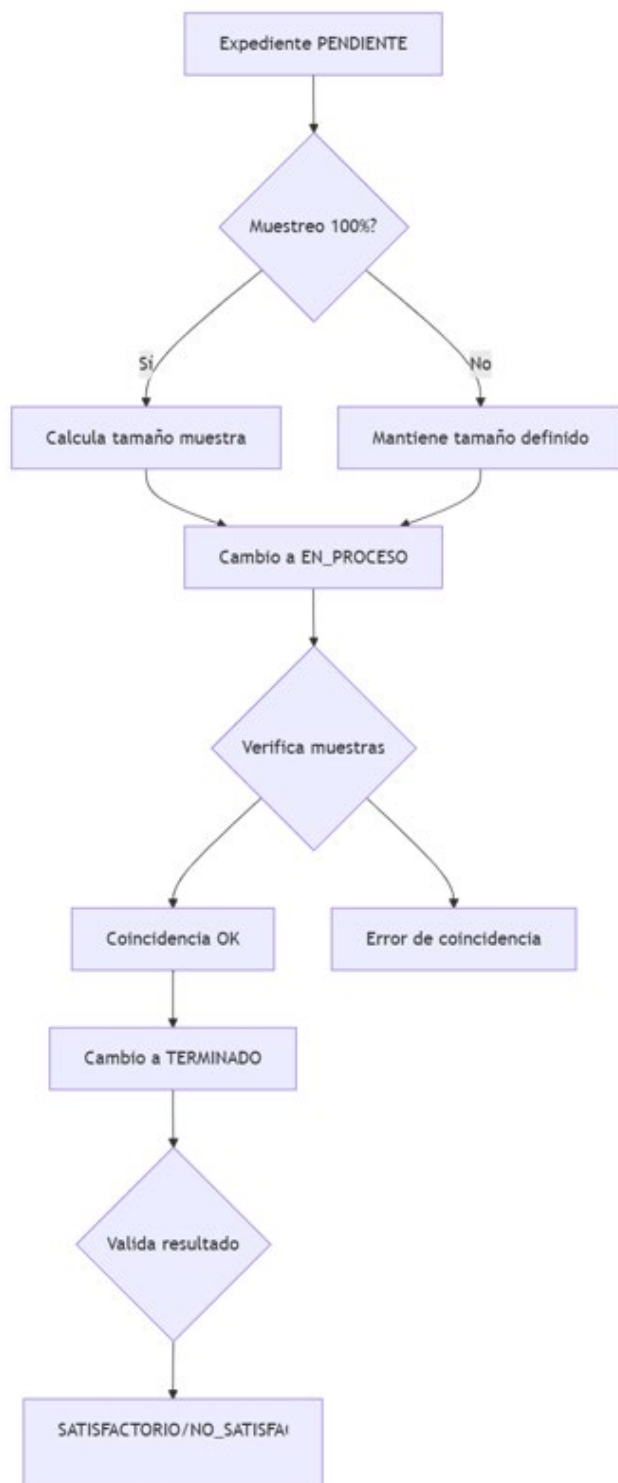
Validaciones Implementadas:

- Tipo de comparación válida: "VALOR_MINIMO", "VALOR_MAXIMO", "RANGO", "NO_COMPARAR"
- Para rangos, valor2 debe ser mayor o igual a valor

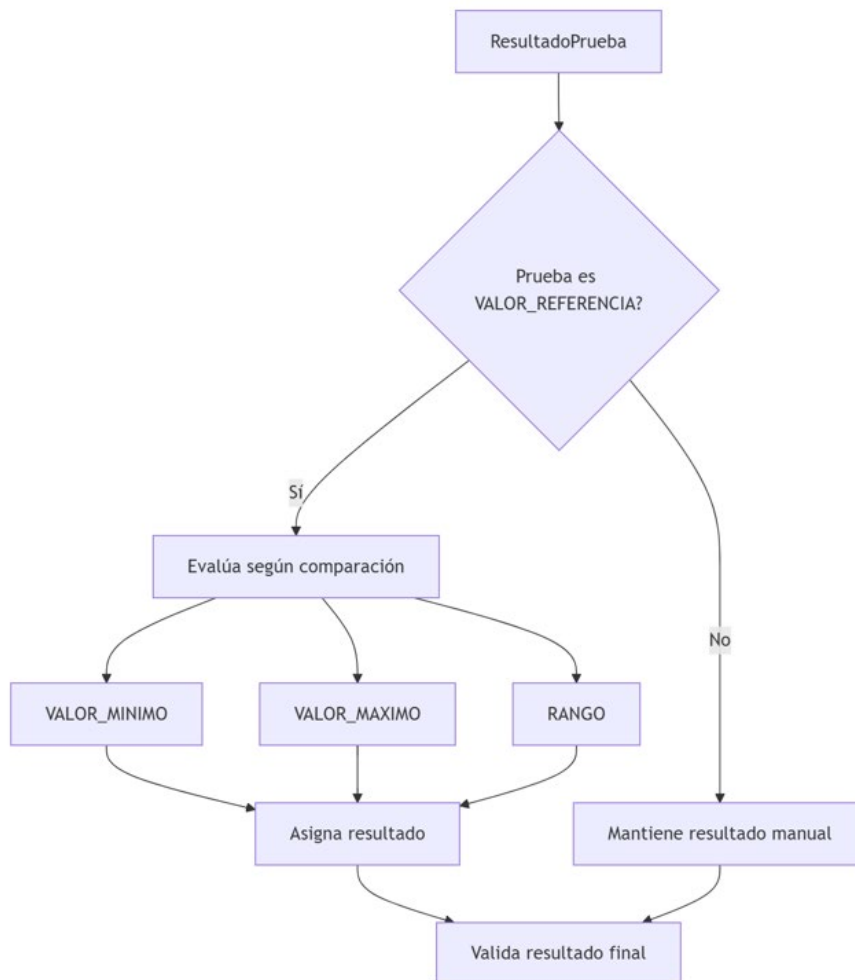


4. Flujos de Validación Complejos

4.1. Flujo de Expediente de Inspección



4.2. Flujo de Resultado de Prueba



5. Manejo de Excepciones

5.1. Tipos de Excepciones Utilizadas

- `ArgumentException`: Para validaciones de parámetros y lógica de negocio
- `ArgumentOutOfRangeException`: Para valores fuera de rangos permitidos
- `IndexOutOfRangeException`: Para valores no contenidos en listas permitidas
- `Exception`: Para validaciones generales de negocio

5.2. Mensajes de Error

Todos los mensajes de error son descriptivos e incluyen:

- Valor actual que causó el error
- Valores esperados o permitidos
- Contexto específico de la validación



6. Mejores Prácticas Implementadas

6.1. Coherencia de Estado

- Validaciones condicionales según el estado del objeto
- Transiciones de estado validadas correctamente

6.2. Validaciones Complejas

- Verificaciones de integridad referencial entre objetos relacionados
- Cálculos automáticos cuando aplica
- Validaciones cruzadas entre múltiples propiedades

6.3. Mantenibilidad

- Listas de valores permitidos centralizadas
- Mensajes de error claros y informativos
- Código organizado por condiciones de negocio

